

---

# Student Course Management System

## Project Report

*Submitted by*

**Yahia Ahmed Mohamed**

Student ID: 11203

Specialization: AI-Based Software Development

Course: Foundations of Software Development

Instructor: Dr. Mostafa Ashraf

[Guthub](#)

2025

---

## Table of Contents

---

| Section | Title                          | Page |
|---------|--------------------------------|------|
| 1       | Introduction                   | 3    |
| 2       | Detailed Project Description   | 3    |
| 3       | Project Beneficiaries          | 4    |
| 4       | Analysis                       | 4    |
| 5       | Task Breakdown Structure       | 5    |
| 6       | Time Plan                      | 6    |
| 7       | System Architecture and Design | 6    |
| 8       | Testing Scenarios and Results  | 7    |
| 9       | End-User Guide                 | 9    |
| 10      | Conclusion                     | 10   |
| 11      | References                     | 10   |

---

## 1. Introduction

---

This report is about the Student Course Management System. The Student Course Management System is a C++ console application. I made it for the Foundations of Software Development course.

The main idea of this project is to use C++ programming ideas. These ideas include Student Course Management System. Making programs that work like objects. I also used a set of tools called the Standard Template Library and thought about how to solve problems with algorithms. I wanted to solve a problem. The problem is to manage students and the courses they are taking. I made a menu that users can interact with.

The Student Course Management System lets users do things. They can add students. Remove students. They can look for a student by their student ID. They can show all the students who are registered. They can put students in courses. They can look at the courses students are taking. They can even sort students by their grades. From a menu, on the console.

## 2. Detailed Project Description

---

The Student Course Management System is a computer program. It helps manage student records and course enrollments. This system is built using the C++ programming language. It has a menu that you can use to interact with it. The Student Course Management System makes it easy to handle student information and course details, in a school setting.

### 2.1 Core Functionality

The system has a menu with numbers that you can use to do things:

1. You can add a student to the system. To do this you need to give the system the students ID, name, email and GPA.
2. You can remove a student from the system. You do this by using the students ID.
3. You can search for a student by using their ID and the system will show you the students details.
4. If you want to see all the students in the system you can use this option. It will show you all the students and their information.
5. You can enroll a student in a course. You need to know the course code to do this.
6. You can also see all the courses that a student is taking.
7. The system can show you all the students. Sort them by their GPA. It will list the students, from the highest GPA to the lowest GPA.
8. When you are done using the system you can exit the application. It will close properly.

---

## 2.2 File Structure

| File                                        | Role                                                                                                                                 |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Person.h / Person.cpp                       | Base class — which's the base class. This base class has some things like name and email.                                            |
| Student.h / Student.cpp                     | Inherits from Person — get these things from Person. They also have some things like student ID, GPA and the courses they are taking |
| Instructor.h / Instructor.cpp               | Inherits from Person — also get the things from Person. They have some things like employee ID and the department they work in       |
| Course.h / Course.cpp                       | to store the course code and the course name.                                                                                        |
| StudentController.h / StudentController.cpp | Manages all student and course operations                                                                                            |
| main.cpp                                    | is where the program starts. It has a menu that the user can use to interact with the program.                                       |

---

### 3. Project Beneficiaries

---

This system helps people, in a school setting:

- Students can see their classes check their GPA and confirm if they are enrolled.
- Academic Administrators can easily update student information add or remove students and manage class enrollments without needing software.
- Instructors can look at class enrollment numbers and student GPA rankings.
- Students learning software development can use this project to learn about Object Oriented Programming using the Standard Template Library and designing C++ code.

### 4. Detailed Analysis

---

#### 4.1 OOP Concepts Applied

| Concept       | Where Applied       | Description                                                                                                                                                                            |
|---------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Encapsulation | All classes         | So what does this mean for all classes? It means that all attributes are private. We can only get to them using getters.                                                               |
| Inheritance   | Student, Instructor | What do these two classes have in common? They both inherit the name and email from the Person class                                                                                   |
| Polymorphism  | display() method    | This display method is virtual in the Person class. It is overridden in the Student and the Instructor classes.                                                                        |
| Abstraction   | StudentController   | What does the StudentController do? It hides all the data management logic, inside the controller class. So when we talk about the StudentController we are talking about abstraction. |

---

## 4.2 STL Containers and Algorithms

| STL Feature                            | Location                                                       | Purpose                                                            |
|----------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------|
| <code>map&lt;int, Student&gt;</code>   | StudentController                                              | Fast $O(\log n)$ student lookup by unique integer ID               |
| <code>map&lt;string, Course&gt;</code> | StudentController                                              | Fast course lookup by string course code                           |
| <code>set&lt;string&gt;</code>         | Student                                                        | Stores course codes — automatically prevents duplicate enrollments |
| <code>vector&lt;Student&gt;</code>     | <code>sortByGpa()</code>                                       | Temporary container used for GPA-based sorting                     |
| <code>sort() + lambda</code>           | <code>sortByGpa()</code>                                       | STL algorithm to sort students by GPA in descending order          |
| <code>find()</code>                    | <code>searchStudent()</code> ,<br><code>removeStudent()</code> | Efficient iterator-based map search                                |

## 4.3 Design Decisions

This system was made with some design choices in mind. The following things helped shape how the system was built:

- I used a map to store student information of a vector. This lets us find a student by their ID fast which is necessary for searching and removing students.
- I used a set to keep track of the courses a student is enrolled in. This helps us avoid adding the course twice without having to write extra code to check for duplicates.
- I kept the parts of the system separate. All the business logic is, in the StudentController, which means the main part of the program can just focus on what the user's doing.
- I made sure the Person class has a destructor. This is important because it makes sure that objects that are based on the Person class are deleted correctly which helps prevent memory leaks.

This system is designed to be efficient and easy to use. These design choices help make that happen.

---

## 5. Task Breakdown Structure

---

The project was divided into the following major tasks:

| Task                           | Description                                                                            | Deliverable                       |
|--------------------------------|----------------------------------------------------------------------------------------|-----------------------------------|
| T1 — Requirements Analysis     | Read and understand assignment specifications, identify required classes and functions | Class list and feature checklist  |
| T2 — UML Design                | Design the class diagram showing inheritance, associations, and attributes             | UML class diagram                 |
| T3 — Base Class Implementation | Implement Person class with protected attributes and virtual functions                 | Person.h / Person.cpp             |
| T4 — Derived Classes           | Implement Student and Instructor classes inheriting from Person                        | Student.h/.cpp, Instructor.h/.cpp |
| T5 — Course Class              | Implement Course class for storing course data                                         | Course.h / Course.cpp             |
| T6 — Controller Implementation | Implement StudentController with all 8 menu operations using STL                       | StudentController.h/.cpp          |
| T7 — Main Menu                 | Implement the interactive menu loop with input validation                              | main.cpp                          |
| T8 — Testing                   | Test all features including edge cases and error handling                              | Test results                      |
| T9 — Documentation             | Write README file and project report                                                   | README.md, Report.docx            |

---

## 6. Time Plan (Gantt Chart)

---

The project was finished in one week. Here is a Gantt chart that shows how the project tasks were spread out over the seven days of the project:

| Task                       | Day 1 | Day 2 | Day 3 | Day 4 | Day 5 | Day 6 | Day 7 |
|----------------------------|-------|-------|-------|-------|-------|-------|-------|
| T1 — Requirements Analysis | X     |       |       |       |       |       |       |
| T2 — UML Design            | X     | X     |       |       |       |       |       |
| T3 — Person Class          |       | X     |       |       |       |       |       |
| T4 — Student & Instructor  |       | X     | X     |       |       |       |       |
| T5 — Course Class          |       |       | X     |       |       |       |       |
| T6 — StudentController     |       |       | X     | X     |       |       |       |
| T7 — Main Menu             |       |       |       | X     |       |       |       |
| T8 — Testing & Fixes       |       |       |       | X     | X     |       |       |
| T9 — GitHub + README       |       |       |       |       | X     |       |       |
| T10 — Report Writing       |       |       |       |       | X     | X     |       |
| T11 — Final Review         |       |       |       |       |       |       | X     |

Note: X indicates the task was actively worked on during that day. Some tasks overlapped to maintain project momentum.



---

## 7. System Architecture and Design

---

### 7.1 Class Hierarchy

The system is made up of layers. At the top is the Person class. This class is important because it gives some information like name and email to the Student and Instructor classes. The StudentController class is, like a manager. It helps to organize the Student and Course objects using containers called maps.

| Class             | Inherits From | Key Attributes                          | Key Methods                                                                               |
|-------------------|---------------|-----------------------------------------|-------------------------------------------------------------------------------------------|
| Person            | —             | name, email                             | getName(),<br>getEmail(), display()<br>[virtual]                                          |
| Student           | Person        | studentId, gpa,<br>courses (set)        | enrollCourse(),<br>showCourses(),<br>display() [override]                                 |
| Instructor        | Person        | employeeId,<br>department               | getEmployeeId(),<br>getDepartment(),<br>display() [override]                              |
| Course            | —             | courseCode,<br>courseName               | getCourseCode(),<br>getCourseName(),<br>display()                                         |
| StudentController | —             | map<int,Student>,<br>map<string,Course> | addStudent(),<br>removeStudent(),<br>searchStudent(),<br>sortByGpa(),<br>enrollInCourse() |

### 7.2 Key Relationships

- The Student and Instructor both inherit from Person. This is an example of an is-a relationship.
- The StudentController is, in charge of all Student and Course objects. This is an example of composition.
- A Student keeps track of course codes. These codes reference Course objects that are managed by the StudentController. This is an example of association.

### 7.3 Data Flow

The user input is received in the main.cpp file. Then it is passed to the StudentController method. This method checks the input to make sure it is correct. Then it does what it needs to do with the information in the internal map containers. After that it shows the result on the console. The thing about this program is that it does not save any information when you are

---

done using it. The StudentController and all its data are in the computers memory while the program is running. When the program is closed all the data, from the StudentController is gone.

## 8. Testing Scenarios and Results

---

The system was tested with scenarios. These tests checked if the system works correctly and is strong. The tests were. They all worked as expected. The system passed all the tests.

### 8.1 Add Student

| Test Case         | Input                                   | Expected Output                |
|-------------------|-----------------------------------------|--------------------------------|
| Add valid student | ID: 101, Sara, sara@email.com, GPA: 3.8 | Student added. :)              |
| Add duplicate ID  | ID: 101, Omar, omar@email.com, GPA: 3.0 | ID already exists!             |
| Invalid GPA       | ID: 104, GPA: 5.0                       | Invalid GPA! Must be 0.0 - 4.0 |

### 8.2 Remove Student

| Test Case                   | Input   | Expected Output        |
|-----------------------------|---------|------------------------|
| Remove existing student     | ID: 102 | Student removed. :(    |
| Remove non-existent student | ID: 999 | Student not found! *_* |

### 8.3 Search Student

| Test Case                   | Input   | Expected Output                |
|-----------------------------|---------|--------------------------------|
| Search existing student     | ID: 102 | Full student details displayed |
| Search non-existent student | ID: 999 | Student not found! *_*         |

### 8.4 Course Enrollment

| Test Case                   | Input                       | Expected Output        |
|-----------------------------|-----------------------------|------------------------|
| Enroll in valid course      | Student: 101, Course: CS101 | Enrolled. :)           |
| Enroll in same course again | Student: 101, Course: CS101 | Already enrolled! @_@  |
| Enroll non-existent student | Student: 999, Course: CS101 | Student not found! *_* |

---

| Test Case                     | Input                       | Expected Output        |
|-------------------------------|-----------------------------|------------------------|
| Enroll in non-existent course | Student: 101, Course: XX999 | Courses not found! +_+ |

## 8.5 Sort by GPA

| Student    | GPA | Expected Rank |
|------------|-----|---------------|
| Sara (101) | 3.8 | 1st           |
| Nour (103) | 3.2 | 2nd           |
| Ali (102)  | 2.5 | 3rd           |

---

## 9. End-User Guide

---

### 9.1 How to Compile

Using g++ (Linux / macOS / Windows with MinGW):

```
g++ -std=c++11 -o app main.cpp Person.cpp Student.cpp Instructor.cpp  
Course.cpp StudentController.cpp
```

### 9.2 How to Run

```
./app
```

On Windows: double-click the compiled .exe file or run it from the command prompt.

### 9.3 Using the Menu

| Option | Action               | Required Input                             |
|--------|----------------------|--------------------------------------------|
| 1      | Add Student          | ID (integer), Name, Email, GPA (0.0 – 4.0) |
| 2      | Remove Student       | Student ID                                 |
| 3      | Search Student       | Student ID                                 |
| 4      | Display All Students | No input required                          |
| 5      | Enroll in Course     | Student ID, Course Code (e.g. CS101)       |
| 6      | Show Student Courses | Student ID                                 |
| 7      | Sort by GPA          | No input required                          |
| 8      | Exit                 | No input required                          |

### 9.4 Important Notes

- The Grade Point Average must be between 0.0 and 4.0. If the Grade Point Average is, outside this range it will not be accepted.
- Each Student ID must be a whole number. The system does not allow Student IDs.
- When you enter a Course code you have to do it right including the capital and small letters. For example you should enter CS101, not cs101.
- Before any Student can enroll in a Course that Course must be added to the system first. This is something that is done in the part of the program.
- All the information is stored in the computers memory. This means that when you close the application you will lose all the data.

---

## 10. Conclusion

---

This project shows how to use Object-Oriented Programming and STL containers to build a working C++ console application. The Student Course Management System does everything it is supposed to do. It does all the things that the assignment said it had to do. Through this project I got to practice and learn about a few important things. These things include making class hierarchies using inheritance and polymorphism keeping data safe with encapsulation choosing the STL containers like map, set and vector and using STL algorithms like sort with lambda to make things work. I also learned how to set up a -file C++ project with the right header guards and separate things that should be separate. The big problem with the system now is that it does not save data when you are done using it. The Student Course Management System has this problem. One thing that could be done to make the Student Course Management System better is to save and load student and course data from files using C++ file I/O streams. This would make the Student Course Management System a lot more useful, in the world.

## 11. References

---

- [GeeksforGeeks website](#)
- [learncpp.com website](#)
- [CodeBro & CodeBeauty channels on Youtube.](#)